

분류 실습

✓ 분류 실습 - 캐글 산탄데로 고객 만족 예측

🔧 Dataset

- 350 피처
- 고객 만족 여부 : 1→불만, 0→만족
- 피처 익명 처리
- 대부분이 만족이고 불만족인 데이터는 일부 → ROC-AUC로 평가

🔧 데이터 전처리

- 필요한 모듈 로딩
- 학습 데이터 DataFrame으로 로딩

```
cust_df = pd.read_csv("train_santander.csv",encoding='latin-1')
```

- Data Shape 확인

```
cust_df.shape
```

(76020, 371) → 371 columns including label

- 만족과 불만족 비율 확인

```
unsatisfied_cnt = cust_df[cust_df['Target']==1].TARGET.count()  
total_cnt = cust_df.TARGET.count()  
print(unsatisfied_cnt/total_cnt)
```

0.04 → 불만족인 고객은 얼마 되지 않는 4%

- .describe()로 각 피처의 값 분포를 확인해 볼때:
 - var3 칼럼 min값이 -999999 → 다른 값에 비해 편차가 심해서 가장 값이 많은 2로 변환
- ID 피처는 드롭 (단순 식별자)
- 피처 데이터 세트와 레이블 세트 분리

```
X_features = cust_df.iloc[:, :-1]
y_labels = cust_df.iloc[:, -1]
```

- 데이터셋을 학습과 테스트로 분리

```
X_train, X_test, y_train, y_test = train_test_split(X_features, y_labels,
                                                    test_size=0.2, random_state=0)
```

- XGBoost의 조기 중단에 들어갈 검증 데이터 세트위해 Train 데이터셋을 다시 쪼갬다

```
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train,
                                            test_size=0.3, random_state=0)
```

XGBoost 모델 학습과 하이퍼 파라미터 튜닝

- 사이킷런 래퍼 XGBClassifier 기반으로 학습
- ROC AUC로 평가
- n_estimators: 500, early_stopping_rounds: 100

```
xgb_clf = XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=0)

xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100, eval_metric='auc',
            eval_set=[(X_tr, y_tr), (X_val, y_val)])

xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:, 1])
print('ROC AUC: {0:.4f}'.format(xgb_roc_score))
```

ROC AUC: 0.8429

- HyperOpt를 이용해 베이지안 최적화 기반으로 XGBoost의 하이퍼 파라미터 튜닝
 - 검색 공간 설정

```
xgb_search_space = {'max_depth': hp.quniform('max_depth', 5, 15, 1),
                    'min_child_weight': hp.quniform('min_child_weight', 1, 6, 1),
                    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.9)}
```

```

        'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)
    }

```

- 목적 함수 설정 : K Fold 교차 검증을 이용한 ROC-AUC 값을 반환 (estimators: 100, early_stopping_rounds: 30)

```

def objective_func(search_space):
    xgb_clf = XGBClassifier(n_estimators=100, max_depth=int(search_space['max_depth']),
                           min_child_weight=int(search_space['min_child_weight']),
                           colsample_bytree=search_space['colsample_bytree'],
                           learning_rate=search_space['learning_rate'])
    roc_auc_list= []
    kf = KFold(n_splits=3)
    for tr_index, val_index in kf.split(X_train):
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]
        xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=30, eval_metric='auc',
                   eval_set=[(X_tr, y_tr), (X_val, y_val)])
        score = roc_auc_score(y_val, xgb_clf.predict_proba(X_val)[:, 1])
        roc_auc_list.append(score)
    return -1 * np.mean(roc_auc_list)

```

- 최적 하이퍼 파라미터 도출

```

best = fmin(fn=objective_func,
            space=xgb_search_space,
            algo=tpe.suggest,
            max_evals=50, # 최대 반복 횟수를 지정합니다.
            trials=trials, rstate=np.random.default_rng(seed=30))
print('best:', best)

```

- 도출된 최적 하이퍼 파라미터를 기반으로 XGBClassifier를 재학습

```

xgb_clf = XGBClassifier(n_estimators=500, learning_rate=round(best['learning_rate'], 4),
                       max_depth=int(best['max_depth']), min_child_weight=int(best['min_child_weight']),
                       colsample_bytree=round(best['colsample_bytree'], 5))

```

```
xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100,
            eval_metric="auc", eval_set=[(X_tr, y_tr), (X_val, y_val)])
xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:,-1])
print('ROC AUC: {0:.4f}'.format(xgb_roc_score))
```

0.8429에서 0.8457로 개선

- 피쳐 중요도 시각화

LightGBM 모델 학습과 하이퍼 파라미터 튜닝

- ROC AUC로 평가
- n_estimators: 500, early_stopping_rounds: 100

```
from lightgbm import LGBMClassifier
lgbm_clf = LGBMClassifier(n_estimators=500)
eval_set=[(X_tr, y_tr), (X_val, y_val)]
lgbm_clf.fit(X_tr, y_tr, early_stopping_rounds=100, eval_metric="auc",
            eval_set=eval_set)
lgbm_roc_score = roc_auc_score(y_test, lgbm_clf.predict_proba(X_test)[:,-1])
print('ROC AUC: {0:.4f}'.format(lgbm_roc_score))
```

ROC AUC: 0.8384

XGBoost 보다 학습에 걸리는 시간 단축

- HyperOpt를 이용해 베이지안 최적화 기반으로 LightGBM의 하이퍼 파라미터 튜닝
 - 검색 공간 설정

```
gbm_search_space = {'num_leaves': hp.quniform('num_leaves', 32, 64, 1),
                    'max_depth': hp.quniform('max_depth', 100, 160, 1),
                    'min_child_samples': hp.quniform('min_child_samples', 60, 120, 1),
                    'subsample': hp.uniform('subsample', 0.7, 1),
                    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)}
}
```

- 목적 함수 설정
 - XGBoost 와 비슷함 : LGBMClassifier 객체를 생성하는 부분만 다름

```

def objective_func(search_space):
    lgbm_clf = LGBMClassifier
    (n_estimators=100, num_leaves=int(search_space['num_leaves'],
        max_depth=int(search_space['max_depth']),
        min_child_samples=int(search_space['min_child
        subsample=search_space['subsample'],
        learning_rate=search_space['learning_rate'])

    ...

```

- 최적 하이퍼 파라미터 도출
 - XGBoost 와 비슷함
- 도출된 최적 하이퍼 파라미터를 기반으로 LGBMClassifier를 재학습

ROC AUC: 0.8446

분류 실습 - 캐글 신용카드 사기 검출

데이터 일차 가공

- 데이터 로딩

```
card_df = pd.read_csv('./creditcard.csv')
```

- 상관 없는 칼럼 드롭

```

def get_preprocessed_df(df=None):
    df_copy = df.copy()
    df_copy.drop('Time', axis=1, inplace=True)
    return df_copy

```

- 데이터 분리 : Train data(70%), Test data(30%)

```

def get_train_test_dataset(df=None):
    df_copy = get_preprocessed_df(df)
    X_features = df_copy.iloc[:, :-1]
    y_target = df_copy.iloc[:, -1]
    X_train, X_test, y_train, y_test = \

```

```
train_test_split(X_features, y_target, test_size=0.3, random_state=0, strata=y_target)
return X_train, X_test, y_train, y_test
```

```
X_train, X_test, y_train, y_test = get_train_test_dataset(card_df)
```

모델 학습/예측/평가

- 로지스틱 회귀

```
# 학습
lr_clf = LogisticRegression()
lr_clf.fit(X_train, y_train)
# 예측
lr_pred = lr_clf.predict(X_test)
lr_pred_proba = lr_clf.predict_proba(X_test)[:, 1]
# 평가
get_clf_eval(y_test, lr_pred, lr_pred_proba)
```

재현율: 0.6216, ROC-AUC: 0.9702

- LightGBM

```
lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1)
get_model_train_eval(lgbm_clf, ftr_train=X_train, ftr_test=X_test, tgt_train=y_train)
```

재현율: 0.7568, ROC-AUC: 0.9790 → 로지스틱 회귀보다 높은 수치

데이터 분포도 변환후 모델 학습/예측/평가

- 중요 피쳐 값의 분포도를 봤을 때 Amount 피쳐(중요한 요소중 하나) → 사용금액이 1000불 이하인 데이터가 대부분이고 26,000불까지 드물지만 많은 금액을 사용한 경우도 있음 → 표준 정규 분포 형태로 바꿔줘야함
- 사이킷런의 StandardScaler클래스를 이용해서 Amount피쳐를 정규 분포 형태로 반환

```
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    scaler = StandardScaler()
```

```
amount_n = scaler.fit_transform(df_copy['Amount'].values.reshape(-1, 1))
df_copy.insert(0, 'Amount_Scaled', amount_n)
df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
return df_copy
```

- 로지스틱 회귀와 LightGBM을 다시 수행
 - 두개 다 정밀도와 재현율이 조금 저하되었지만 큰 성능상의 변경은 없음
- 로그 변환을 이용해서 Amount피처를 정규 분포 형태로 반환

```
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy
```

- 로지스틱 회귀와 LightGBM을 다시 수행
 - 로지스틱 회귀 : 원본 데이터 대비 정밀도 향상, 재현율 저하
 - LightGBM : 재현율 향상

이상치 데이터 제거 후 모델 학습/예측/평가

- 이상치 찾는 방법 여러가지 있지만 IQR(Inter Quantile Range)방식을 적용
 - 사분위
 - Q1: 1/4(25%)
 - Q2: 2/4(50%)
 - Q3: 3/4(75%)
 - Q4: 4/4(100%)
 - IQR : 사분위 값의 편차를 이용하는 기법
 - $IQR = 25\% \sim 75\%$ Q3
- IQR로 이상치 데이터 검출법
 - IQR에 weight(1.5)을 곱해서 생성된 범위로 최댓값과 최솟값을 결정한 뒤 최댓값을 초과하거나 최솟값에 미달하는 데이터를 이상치로 간주

- 최댓값 = $Q3 + (IQR * 1.5)$
 - 최솟값 = $Q1 - (IQR * 1.5)$
- 피쳐들이 많은 경우 → 가장 상관성이 높은 피쳐들로 이상치 검출 하는게 효율적
- `corr()`를 이용해 각 피쳐별로 상관도를 구함

```
import seaborn as sns

plt.figure(figsize=(9, 9))
corr = card_df.corr()
sns.heatmap(corr, cmap='RdBu')
```

결정 레이블(Class)피쳐와 상관관계 가장 높은 피쳐중 하난 V14

- V14에서 이상치를 찾아서 제거

```
def get_outlier(df=None, column=None, weight=1.5):
    fraud = df[df['Class']==1][column]
    quantile_25 = np.percentile(fraud.values, 25)
    quantile_75 = np.percentile(fraud.values, 75)
    iqr = quantile_75 - quantile_25
    iqr_weight = iqr * weight
    lowest_val = quantile_25 - iqr_weight
    highest_val = quantile_75 + iqr_weight
    outlier_index = fraud[(fraud < lowest_val) | (fraud > highest_val)].index
    return outlier_index
```

- 위에 만들어진 `get_outlier()`함수를 이용해서 이상치 추출

```
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    outlier_index = get_outlier(df=df_copy, column='V14', weight=1.5)
    df_copy.drop(outlier_index, axis=0, inplace=True)
    return df_copy
```

- 로지스틱 회귀와 LightGBM을 다시 수행

- 로지스틱 회귀와 LightGBM 모두 예측 성능 크게 향상됨
 - 로지스틱 회귀 : 재현율 60.14%에서 67.12%으로
 - LightGBM: 재현율 76.35%에서 82.88%으로

SMOTE 오버 샘플링 적용 후 모델 학습/예측/평가

- SMOTE 적용할 때 학습 데이터 세트만 오버 샘플링 해야 함 (검증/테스트 까지 하면 올바른 검증/테스트를 할 수 없음)
- fit_resample() 이용

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=0)
X_train_over, y_train_over = smote.fit_resample(X_train, y_train)
```

- SMOTE 적용 전과 후 비교

```
print('SMOTE 적용 전 학습용 피쳐/레이블 데이터 세트: ', X_train.shape, y_train.shape)
print('SMOTE 적용 후 학습용 피쳐/레이블 데이터 세트: ', X_train_over.shape, y_train_over.shape)
print('SMOTE 적용 후 레이블 값 분포: \n', pd.Series(y_train_over).value_counts())
```

적용 후 2배에 가깝게 데이터 증식

- SMOTE을 적용한 데이터로 로지스틱 회귀 모델을 학습/예측/평가
 - 재현율 : 92.47%로 크게 증가, But 정밀도가 5.4%로 급격히 저하
 - 이유: 오버 샘플링으로 인해 실제 원본 데이터의 유형보다 더 많은 1 데이터를 학습하면서 실제에서 예측할 때도 지나치게 1를 맞추면서 정밀도 저하됨
 - 분류 결정 임계값에 따른 정밀도와 재현율 곡선을 통해 SMOTE로 학습한 모델의 문제점 시각화

```
precision_recall_curve_plot( y_test, lr_clf.predict_proba(X_test)[: , 1] )
```

임계값 0.99 이하 : 재현율 좋고 정밀도 낮음

임계값 0.99 이상 : 재현율 낮고 정밀도 좋음

임계값의 민감도가 심해 올바른 재현율/정밀도 성능을 얻을 수 없기 때문에 올바른 예측 모델을 생성하지 못함

- SMOTE를 적용한 데이터로 LightGBM 모델을 학습/예측/평가

```
lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, n_jobs=-1)
get_model_train_eval(lgbm_clf, ftr_train=X_train_over, ftr_test=X_test,
                     tgt_train=y_train_over, tgt_test=y_test)
```

재현율이 높아졌지만 정밀도는 낮아짐

- 전체적으로, 정밀도 보다 재현율을 높이고 싶은 모델이 경우 SMOTE를 적용
-